Estimating Arrival Delay and Delay Propagation in Multi-Leg Shipments with Gradient-Boosted Models

Anchal Tandekar. Aiqon AI/SE Intern Take-Home Report. All figures are derived from the accompanying repository (README.md, LLD.md, src/, outputs/).

Abstract. This report addresses the prediction of delivery delay for multi-leg shipments at successive points in their journey. A synthetic generator with explicit delay propagation and five planted causes produces 300 shipments, which are evaluated at seven checkpoints (C0 to C6). A LightGBM regressor (Model R) predicts remaining delay, a quantile model (Model Q) provides an 80% interval, and a classifier (Model C) estimates the probability of a delay above 6 h. Across five split seeds, Model R outperforms the strongest baseline (delay observed so far) at six of seven checkpoints, with skill scores of 0.36 to 0.41 after port arrival. Grouped TreeSHAP explanations recover most planted causes, although attribution of missed hub connections (ROC-AUC 0.742) falls short of the 0.8 target.

1. Introduction

The task is, given a shipment's route, schedule and the milestones observed so far, to estimate (i) the delay at delivery, (ii) an interval for that delay, and (iii) the probability that the shipment is late, together with an explanation of each prediction. Every shipment is replicated at checkpoints C0 (no milestones observed) to C6 (after customs), so that a single model serves all stages of the journey. All models are LightGBM and are evaluated on held-out shipments over five split seeds (42 to 46), using a 20% shipment-level test split and five-fold grouped cross-validation.

2. Data Generation and Delay Propagation

The generator (`src/generate.py`) produces 300 shipments over 19 route templates from seed 42; 55% of shipments connect through a single hub. Each stage inherits the lateness of its predecessor, absorbs a fraction of it, and adds its own delay:

$$\text{delay}_{\text{new}} = \text{delay}_{\text{in}} \cdot (1 - \text{absorb}) + \text{own delay}$$

The hub is the exception: the cargo either makes the connection, when the buffer covers the incoming lateness, or misses the sailing, in which case the delay is reset to the wait for the next departure. Five causes are planted: missed connection, missed berth slot, congestion above the port norm, rare storms on a single leg, and absorption. Penalty magnitudes are hidden from the model, which is given only two operational constants (a 12 h hub connection time and a 12 h berth window). Hidden causes are written to `truth.csv` and are used solely for evaluation and explanation. Missing logs are simulated independently (default rate 0.10, affecting only `act_port_done` and `act_customs`), and a gap rule flags added delay that spans a hidden stage. A shipment is labelled delayed above a 6 h threshold, which yields 60% delayed shipments at the default seed (LLD.md); the classes are therefore not severely imbalanced.

3. Feature Construction

The feature set comprises: route attributes (journey, origin, hub, destination, connecting flag, legs); plan attributes (planned total and leg times, hub buffer, nominal slack, scheduled day of week and day of year); port norms (base dwell, customs, congestion); observed stage delays and added delays, `delay_so_far` and the number of observed stages; environmental variables (leg weather, hub and port congestion, arrival day); link features (`slack_remaining`, `slot_margin`, congestion multiplied by dwell and customs, port-completion weekday and weekend flag); and missingness indicators. A single visibility map in `checkpoints.py` governs which features are available at each checkpoint, which prevents leakage; for example, realised port congestion is never used before C4.

4. Models

- **Baselines.** B0 predicts zero delay, B1 predicts the journey mean over training shipments, and B2 predicts `delay_so_far`.
- **Model R.** An L1 LightGBM regressor on `remaining_delay_h`, whose output is added to `delay_so_far`. Model Q (0.1 and 0.9 quantiles) supplies the prediction interval.
- **Model C.** A delay classifier (threshold 6 h), selected from a four-candidate grid by shipment-grouped cross-validated log loss (`outputs/metrics/model_c_tuning.csv`): `small` (depth 3, 4 leaves, learning rate 0.05, minimum child samples 15), `medium` (depth 4, 8 leaves, 0.05, 15), `regularized` (depth 3, 4 leaves, 0.05, 30) and `faster_learning` (depth 4, 8 leaves, 0.1, 15). `regularized` was selected on four of five split seeds (CV log loss 0.455 to 0.492) and `small` on

seed 44 (0.497); `faster_learning` performed worst on every seed. Probabilities are not calibrated post hoc.

5. Results

Table 1 reports held-out means over five split seeds. The skill score is defined as $1 - \mathrm{MAE}_{\mathrm{R}} / \mathrm{MAE}_{\mathrm{B2}}$.

Table 1. Held-out performance by checkpoint.

Ckpt	Model R MAE (h)	B2 MAE (h)	Skill	R beats B2 (seeds)	Model C ROC-AUC	Model C Brier	Q coverage (nominal 0.8)
C0	14.72 ± 1.15	17.55	0.16	5/5	0.56	0.251	0.72
C1	14.50 ± 0.88	16.74	0.13	5/5	0.56	0.254	0.69
C2	12.13 ± 0.95	15.20	0.20	5/5	0.69	0.227	0.71
C3	9.28 ± 2.29	9.25	-0.003	2/5	0.73	0.209	0.75
C4	4.64 ± 0.41	7.27	0.36	5/5	0.90	0.124	0.67
C5	3.28 ± 0.40	5.56	0.41	5/5	0.94	0.096	0.64
C6	2.42 ± 0.25	4.13	0.41	5/5	0.96	0.078	0.77

Sources: `model_r_vs_b2.csv`, `model_c_classification_metrics.csv` and `model_q_coverage.csv` (all in `outputs/metrics/`).

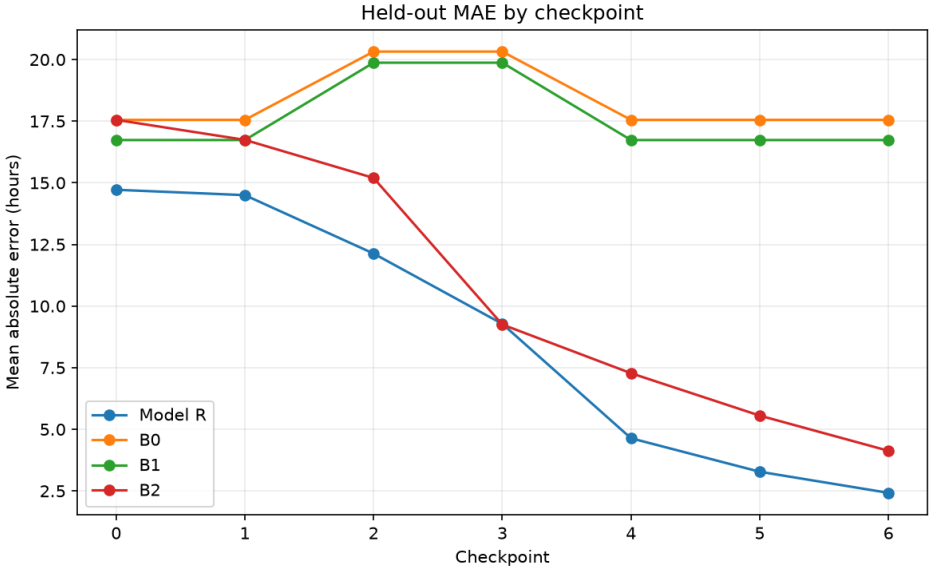


Figure 1. Held-out MAE by checkpoint for B0, B1, B2 and Model R (*`outputs/metrics/model_r_vs_baselines.png`*).

Model R outperforms B2 on all five seeds at every checkpoint except C3 (after hub departure), where the two are effectively tied (skill -0.003, Model R ahead on two of five seeds): once the hub outcome is known, `delay_so_far` is already a strong estimator. The improvement is largest after port arrival (skill 0.36 to 0.41), where the slot and congestion features become informative. B0 and B1 remain between approximately 16.7 and 20.3 h MAE at every checkpoint (read from Figure 1). Model C discriminates poorly before the hub (ROC-AUC 0.56) and well from port arrival onward (0.90 or above). Model Q intervals under-cover (0.64 to 0.77 against a nominal 0.8), with no quantile crossings.

Subgroup analysis (`model_r_by_group.csv`, C0). Model R achieves 16.74 h MAE on connecting shipments against 20.32 h for B2, 12.07 h against 13.93 h on direct shipments, and 17.26 h against 22.40 h on shipments with a hidden stage. Model R outperforms B2 on all five seeds in each subgroup.

Feature ablation (`model_r_ablations.csv`, mean MAE over C0 to C6). Configuration A0 (route, plan and observed delays) yields 9.145 h, A1 (A0 plus environment) 8.821 h, and A2 (A1 plus link features) 8.709 h. A2 improves on A0 by 0.436 h (4.8%), and mean skill rises from 0.182 to 0.240. The gains are most pronounced at C2 and C4 to C6 (at C4, 6.26 h for A0 against 4.64 h for A2). At C3, A2 (9.277 h) is marginally worse than B2 (9.251 h).

Sensitivity to missing logs (`model_r_missingness_sweep.csv`, identical world and split seeds). Mean Model R MAE is 8.585 h at 0% missingness, 8.709 h at 10%, 8.707 h at 20% and 8.784 h at 30%, an

increase of approximately 0.20 h (2.3%) from 0% to 30%. Skill remains between 0.235 and 0.240, indicating that Model R retains its advantage over B2 as logs become incomplete.

The ablation, missingness and SHAP results were obtained from a full `run_all` with default settings (seed 42, 300 shipments, five split seeds, missing rate 0.1); its Model R and B2 figures agree exactly with Table 1.

6. Explanation Analysis

Grouped TreeSHAP was computed for 1,840 held-out Model R predictions (`src/explain.py`). The largest mean absolute group contributions are attributed to the hub (3.244 h), dwell and congestion (2.076 h), and route and plan (1.936 h). Slack behaves as expected: across 850 rows with visible slack, slack and its SHAP contribution have a correlation of -0.555, so that negative slack increases predicted remaining delay and positive slack decreases it.

Agreement with the planted causes (`model_r_explanation_recovery.csv`) is mixed. Hub SHAP values separate missed connections at C2 with ROC-AUC 0.742 (170 shipments, 44 missed), below the 0.8 target specified in the design. The slot group contributes 0.462 h more for missed berth slots than for kept slots at C4, satisfying the check. The storm leg is the highest-ranked observed factor in 100% of storm cases (32 on leg 1, 8 on leg 2). These checks demonstrate that the explanations recover the generator's planted structure; they do not establish real-world causality.

7. Limitations and Future Work

- The data is synthetic. The results show that the method recovers planted structure, not real port behaviour, and the generator noise is not calibrated against real data.
- The sample of 300 shipments is small; means and dispersion over five split seeds are therefore reported in place of single estimates.
- Missingness is simulated as random, whereas real gaps are generally not. Each journey contains at most one hub.
- The optional adjusted-slack feature (hub congestion) is disabled because the LLD specifies no formula for it.
- Model C is uncalibrated and Model Q under-covers. Future work comprises probability calibration, conformal interval widening, improved hub attribution (C2 recovery is below target), and evaluation on real or calibrated data.

8. Reproducibility

`make all` executes data generation, feature construction, splitting, baselines, Models R, Q and C, evaluation, the missingness sweep, explanations and ablations, and writes `outputs/run_manifest.json` containing the seed, configuration, Git revision and headline metrics. Individual stages are available as `make data`, `features`, `splits`, `baselines`, `train` (Model R), `eval`, `train-q`, `eval-q`, `train-c`, `eval-c`, `eval-missingness`, `explain`, `test` and `clean`. Default settings (`src/config.py`) are: seed 42, 300 shipments, delay threshold 6 h, missing rate 0.10, test fraction 0.20, five cross-validation folds and five split seeds. The full run underlying Sections 5 and 6 recorded these settings in its manifest.